

# 01-编译器概览与词法分析复习讲义

## 目录

|   |                   |
|---|-------------------|
| 1 | 01 编译器概览与词法分析复习讲义 |
|---|-------------------|

|   |
|---|
| 2 |
|---|

# 1 01 编译器概览与词法分析复习讲义

对应资料：

- Slides/Slides/Ch1-Introduction to Compilers.pdf
- Slides/Slides/Ch3-1-Finite Automata.pdf
- Slides/Slides/Ch3-2-Lexical Analysis.pdf
- Slides/Slides/handout/1 Regex NFA DFA.pdf
- Review/compile.pdf
- Exam/2025.pdf 词法分析题
- Exam/过时/2022-exam-A.pdf 题目 1
- Exam/过时/2023-exam-A.pdf 题目 1
- Exam/过时/2021-exam-A.pdf 题目 1

## 1.1 一、先记住考试目标

词法分析题稳定考三步：

正则表达式 -> Thompson NFA -> 子集构造 DFA -> DFA 最小化

零基础记法：

- 正则表达式描述一类字符串。
- NFA 像“分身试路”，同一输入能走多条路，还能走 epsilon 边。
- DFA 像“把全部分身装进一个集合”，每个 DFA 状态就是一组 NFA 状态。
- 最小化就是合并“之后对所有输入表现相同”的 DFA 状态。

## 1.2 二、编译器总流程

编译器把源程序翻译为目标程序。阶段顺序：

```

Source Code
-> Lexer
-> Tokens
-> Parser
-> AST / Parse Tree
-> Semantic Analysis
-> IR
-> Optimization
-> Target Code Generation
-> Machine Code

```

每阶段一句话：

- Lexer: 字符流变 token 流。
- Parser: token 流变语法树。
- Semantic Analysis: 查类型、作用域、声明使用。
- IR: 生成中间表示。
- Optimization: 改写 IR, 保持语义不变。
- Code Generation: 生成目标机器代码。

### 1.3 三、字符串、语言、正则

必须会写：

```

epsilon : 空串
L1 L2   : {xy | x in L1, y in L2}
L*      : L 重复 0 次或多次
L+      : L 重复 1 次或多次
Lbar    : Sigma* 中不属于 L 的串
L~R     : L 中所有串反转

```

正则表达式语义：

```

emptyset : 空语言
epsilon  : {epsilon}
a        : {a}
A | B    : 并
AB       : 连接
A*       : Kleene star

```

易错点：

- $\{\epsilon\}$  不是空集。
- $A^+ = A A^*$ 。
- 正则语言对并、交、连接、补、反转、Kleene star 封闭。

## 1.4 四、DFA 与 NFA 定义

DFA:

```
M = (Q, Sigma, delta, q0, F)
delta : Q x Sigma -> Q
```

DFA 接受条件:

读完整个串后，最终状态在  $F$  中。

NFA:

```
M = (Q, Sigma, delta, q0, F)
delta : Q x (Sigma union {epsilon}) -> 2^Q
```

NFA 接受条件:

读完整个串后，能到达的状态集合与  $F$  有交集。

## 1.5 五、Thompson 构造

按正则结构从内到外画。考试直接套模板。

字符:

```
start --a--> end
```

epsilon:

```
start --epsilon--> end
```

选择  $A \mid B$ :

```

      epsilon      A      epsilon
start -----> A.start ... A.end ----->
      \
      \ epsilon      B      epsilon /
      -----> B.start ... B.end ----->

```

连接 AB:

```
A.start ... A.end --epsilon--> B.start ... B.end
```

闭包 A\*:

```
start --epsilon--> A.start ... A.end --epsilon--> A.start
|                                     |
+-----epsilon-----> end
                A.end --epsilon--> end
```

答题顺序:

1. 给每个子表达式画小 NFA。
2. 按连接、选择、闭包组合。
3. 给状态编号。
4. 起点画入箭头，终点画双圈。

## 1.6 六、子集构造：NFA 到 DFA

两个操作:

`epsilon-closure(S)`: 从 S 出发只走 `epsilon` 边能到达的状态集合  
`move(S, a)`: 从 S 出发读字符 a 能到达的状态集合

算法模板:

```
Start = epsilon-closure({NFA start})
把 Start 作为第一个 DFA 状态

while 还有未处理的 DFA 状态 S:
    对每个输入字符 a:
        T = epsilon-closure(move(S, a))
        记录 S --a--> T
        若 T 第一次出现, 把 T 加入 DFA 状态集

若 S 中含 NFA 终态, 则 S 是 DFA 终态
```

表格写法:

| DFA 状态 | NFA 状态集合 | on a | on b | 是否终态 |
|--------|----------|------|------|------|
| A      | {0,1,2}  | B    | C    | 否    |
| B      | {1,3,4}  | B    | D    | 是    |

## 1.7 七、DFA 最小化

核心判断：

两个状态能合并，当且仅当从它们出发读任意后续输入，接受/拒绝结果都一致。

算法模板：

$P0 = \{\text{终态集合}, \text{非终态集合}\}$

重复：

对每一组  $G$ ，检查组内状态在每个输入字符上的后继落到哪个组。

后继组模式不同，就拆开  $G$ 。

分组不再变化时，每个组变成最小 DFA 的一个状态。

最小化表格写法：

初分组： $\{A, C\}, \{B, D\}$

按  $a$  检查： $A \rightarrow B, C \rightarrow B$ ，同组； $B \rightarrow D, D \rightarrow D$ ，同组

按  $b$  检查： $A \rightarrow A, C \rightarrow C$ ，同组； $B \rightarrow C, D \rightarrow C$ ，同组

分组不再变化，得到两个状态。

## 1.8 八、正则语言泵引理

用于证明语言不是正则语言。

形式：

若  $L$  是正则语言，则存在常数  $C$ 。

对任意  $s \in L$  且  $|s| \geq C$ ，

存在  $s = xyz$ ，满足  $|xy| \leq C, |y| > 0$ ，

并且对任意  $i \geq 0, xy^i z \in L$ 。

考场证明模板：

假设  $L$  是正则语言，取泵长度  $C$ 。

选一个长串  $s \in L$ 。

对任意分解  $s = xyz$ ，且  $|xy| \leq C, |y| > 0$ 。

由  $|xy| \leq C$  得  $y$  落在某一段固定字符中。

取  $i = 0$  或  $i = 2$ ，得到的新串不在  $L$ 。

矛盾，所以  $L$  不是正则语言。

## 1.9 九、真题对应

2025:

题型：正则表达式  $\rightarrow$  NFA  $\rightarrow$  DFA  $\rightarrow$  最小化。

答法：按第五、六、七节模板写，必须标明 NFA 状态集合对应的 DFA 状态。

2023:

正则： $a((() \mid [])^*)$

任务：Thompson NFA；子集构造 DFA。

重点：() 和 [] 是输入串中的两个后缀 token，不是正则元符号。

2022:

正则： $((\epsilon \mid a)b^*)^*$

任务：Thompson NFA；子集构造 DFA；DFA 最小化。

2021:

正则： $\epsilon \mid (a^*b)$

任务：Thompson NFA；DFA；最小化。

## 1.10 十、自测题

题 1： $A^+$  如何改写成只含  $*$  的正则？

答案：

$$A^+ = A A^*$$

题 2：DFA 与 NFA 接受能力哪个强？

答案：

一样强。二者都识别正则语言，NFA 能用子集构造转成等价 DFA。

题 3：子集构造中，DFA 的终态怎么判定？

答案：

某个 DFA 状态是 NFA 状态集合  $S$ 。

若  $S$  与 NFA 终态集合  $F$  有交集，则该 DFA 状态是终态。

题 4：DFA 最小化第一步怎么分组？

答案：

先分成终态集合和非终态集合。